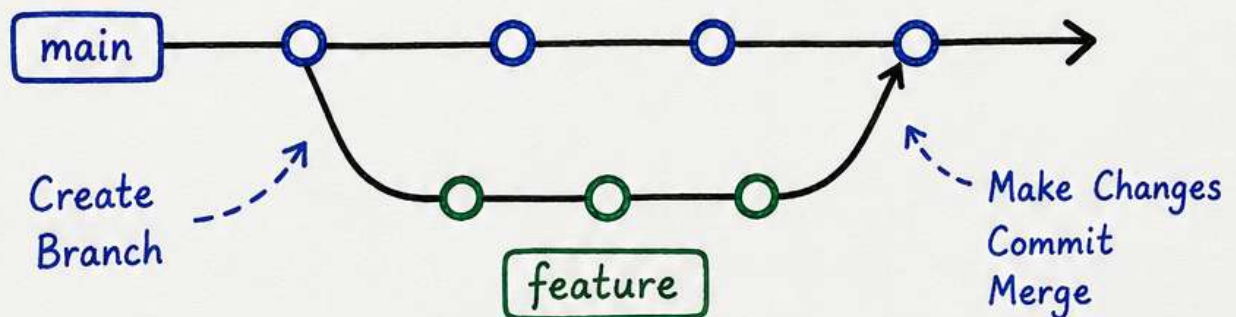


20 GIT INTERVIEW SCENARIOS

Every Developer Should Know



- ★ 20 Most Important Git Scenarios
- ✓ Clear Explanation + Diagrams
- 🎯 Perfect for Interviews & Real-world



Comment "GIT" to get
the complete PDF ❤️

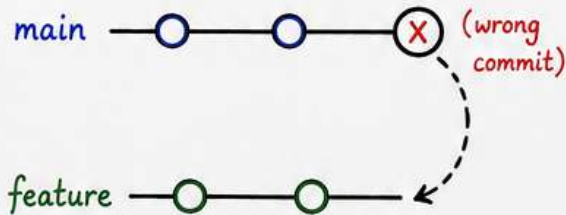


1

SCENARIO: You committed to **wrong branch**.
How to move it to **correct branch**?

EXAMPLE

You meant to commit on 'feature' but did it on 'main' by mistake.

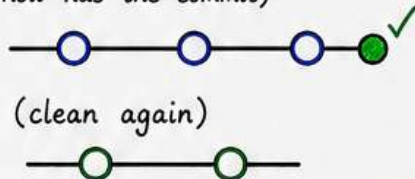
SOLUTION

- ① `git checkout main`
switch to correct branch
- ② `git cherry-pick <commit-hash>`
pick the commit to main
- ③ `git checkout feature`
go back to feature branch
- ④ `git reset --hard HEAD~1`
remove the commit from feature

RESULT

main (now has the commit)

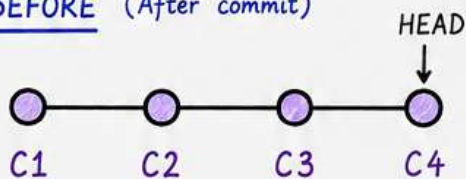
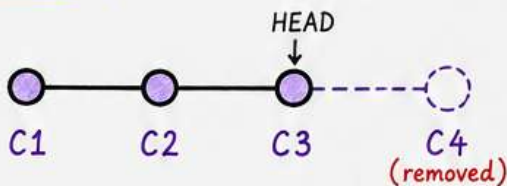
feature (clean again)

TIP

- cherry-pick copies the commit to another branch.
- `reset --hard` removes it from the current branch.

2

SCENARIO: You made the last commit, but want to remove it and **discard changes**.

BEFORE (After commit)AFTER (After `reset --hard HEAD~1`)SOLUTION

```
git reset --hard HEAD~1
```

- Moves **HEAD** back by 1 commit.
- Discards all changes in working directory and staging area.
- Nothing is **left**.

NOTE

This action is **permanent**.
You cannot get back the commit and changes.

COMMAND QUICK VIEW

```
git checkout main → git cherry-pick <hash> →  
git checkout feature → git reset --hard HEAD~1
```



3

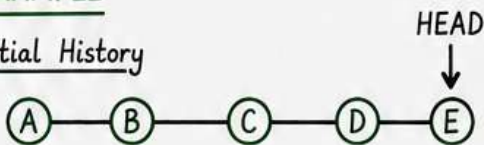
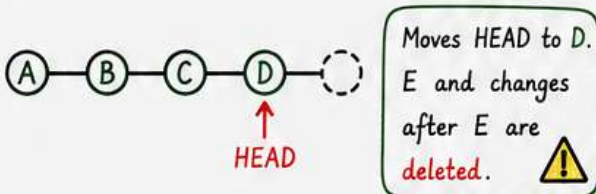
SCENARIO: What is the difference between `git reset` and `git revert`?

ANSWER

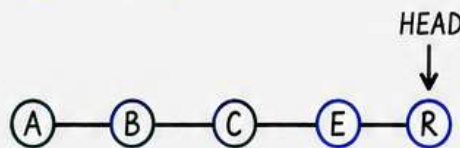
- `reset` moves the branch pointer and can **delete** history.
- `revert` creates a new commit that **undoes** the changes.

EXAMPLE

Initial History

① Using `git reset --hard HEAD~1`KEY DIFFERENCE

`reset` → **Dangerous** (history can be lost)
`revert` → **Safe** (history is preserved)

② Using `git revert E`

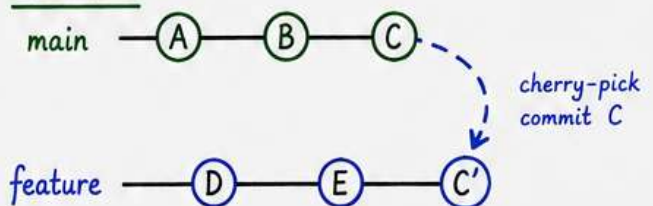
Creates new commit (R) that undoes changes of E. History is safe. ✓

4

SCENARIO: Cherry-pick a commit from one branch to another.

ANSWER

`git cherry-pick <commit-hash>` copies the specified commit and applies it to the current branch.

EXAMPLESTEPS

- ① Checkout target branch
`git checkout feature`
- ② Cherry-pick the commit
`git cherry-pick <commit-hash>`
- ③ The commit will be copied with new commit id (C').

TIPS

- ✓ Use when you need specific changes from another branch.
- ✓ Very useful for bug fixes.
- ✓ If conflict occurs, resolve and run:
`git cherry-pick --continue`

COMMAND QUICK VIEW

<code>git cherry-pick <hash></code>	→	copy commit to current branch
<code>git cherry-pick --continue</code>	→	continue after resolving conflict
<code>git cherry-pick --abort</code>	→	cancel cherry-pick

5

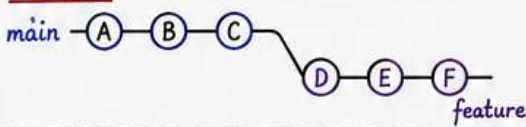
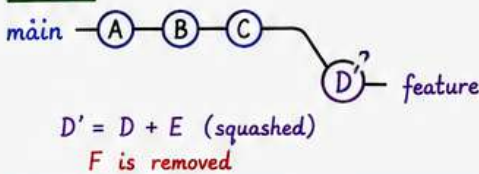
SCENARIO: What is `git rebase -i` and when to use it?

ANSWER

`git rebase -i` (interactive rebase) is used to **edit**, **squash**, **reorder**, or remove commits in a branch. It rewrites history, so use it on **local branches** (not on shared branches).

GOAL

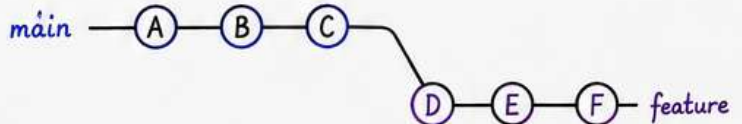
1. Squash E into D
2. Remove F (not needed)

BEFOREAFTERWHEN TO USE

- ✓ Clean up messy commit history
- ✓ Squash small/fixup commits
- ✓ Reorder commits
- ✓ Remove unwanted commits
- ✗ Don't use on public/shared branches

EXAMPLE

Suppose we have this history on branch 'feature':

STEPS

- ① Start interactive rebase for last 3 commits:

```
$ git rebase -i HEAD~3
```

- ② Editor opens with something like this:

pick D	commit D	} (oldest at bottom)
pick E	commit E	
pick F	commit F	

- ③ Change it to:

pick D	commit D	↗ squash = merge into previous commit ↘ drop = remove commit
squash E	commit E	
drop F	commit F	

- ④ Save & exit. Git will rewrite history.

NOTE

Rebase -i changes commit IDs.
Always force push if the branch was pushed before.
`git push --force-with-lease`

6

SCENARIO: How to **squash** multiple commits into one?

ANSWER

Squashing combines multiple commits into a single clean commit.

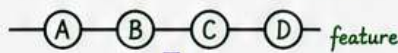
It helps to keep history **simple** and **readable**.

WHEN TO USE

- ✓ When working on a feature
- ✓ before merging to main
- ✓ To keep history clean
- ✓ Avoid clutter

EXAMPLE

We have 4 commits on 'feature':



After squash into 1 commit:

TIP

Squash before merging, not after pushing.

STEPS

- ① Start interactive rebase:

```
$ git rebase -i HEAD~4
```

- ② In editor, keep first as 'pick' and change others to 'squash'.

- ③ Save & exit.

- ④ One editor opens to combine commit messages.

- ⑤ Save & exit. You'll have one clean commit.

COMMAND QUICK VIEW

`git rebase -i HEAD~N` → start interactive rebase
`pick / squash / drop` → choose action

`git push --force-with-lease`
 → update remote safely



7

SCENARIO: What is the difference between git merge and git rebase?

ANSWER

Both integrate changes from one branch into another, but in different ways.

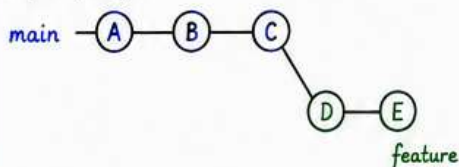
- **merge** creates a new commit.
- **rebase** rewrites history by placing your commits on top of the target branch.

KEY DIFFERENCE

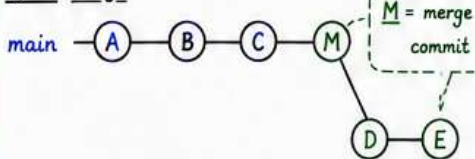
Aspect	git merge	git rebase
Creates new commit	Yes	No
History	Preserved (with merge commit)	Rewritten (linear)
Safe on shared branch	Yes	No (avoid)
Best used when	Collaborating	Keeping history clean
Result	Non-linear history	Linear history

EXAMPLE: git merge

Before merge



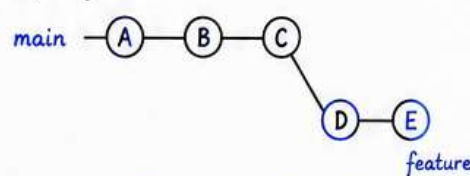
After merge



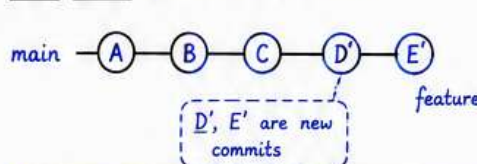
- ★ Merge keeps all history and creates a merge commit.

EXAMPLE: git rebase

Before rebase



After rebase (feature on top of main)



- ★ Rebase rewrites history and keeps it linear.

TIP

- Use merge for team collaboration.
- Use rebase for clean, readable history (before pushing).

8

SCENARIO: How to resolve merge conflicts?

ANSWER

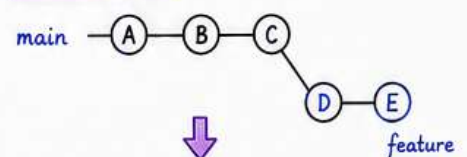
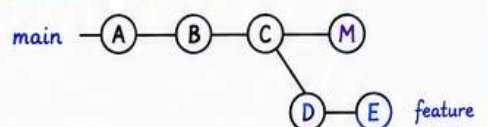
When two branches modify the same line, Git cannot decide automatically. You must resolve it manually.

CONFLICT EXAMPLE

```
<<<<<< HEAD (main)
int value = 10;
=====
int value = 20;
>>>>>> feature
```

STEPS TO RESOLVE

- ① Run merge
\$ git merge feature
- ② Git shows conflicts in files.
- ③ Open the file and edit the conflict.
Keep the changes you want.
- ④ Mark as resolved
\$ git add <file>
- ⑤ Commit the merge
\$ git commit
or git commit -m "Resolved merge conflict"

BEFORE MERGEAFTER MERGE (conflict resolved)NOTE

- Always test after resolving conflicts.
- Communicate with your team.
- Never delete others' changes blindly.

COMMAND QUICK VIEW

git merge <branch>
→ start merge

git status
→ shows conflicted files

git add <file>
→ mark as resolved

git commit
→ complete merge

9

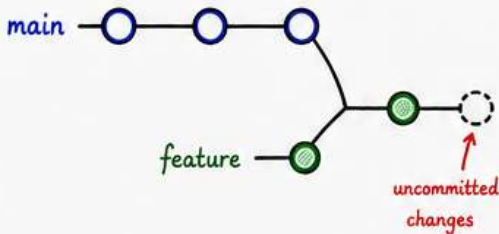
SCENARIO: How to use `git stash` and `git stash pop`?

ANSWER

Stash temporarily saves your uncommitted changes so you can switch branches or work on something else. Later you can bring them back using `stash pop`.

EXAMPLE

You are on 'feature' branch and made some changes (not committed).

SOLUTION

- 1 Stash your changes
\$ `git stash` # save changes to stash
- 2 Switch branch
\$ `git checkout main`
work on something else
- 3 Go back to feature branch
\$ `git checkout feature`
- 4 Bring back your changes
\$ `git stash pop`
apply and remove stash
- 5 (Optional) See stash list
\$ `git stash list`

TIP

- `git stash pop` applies and removes the stash.
- Use `git stash apply` if you want to keep the stash.
- Use `git stash list` to see all stashes.

QUICK FLOW

```

git stash
↓
checkout other branch
↓
checkout feature
↓
git stash pop
  
```

10

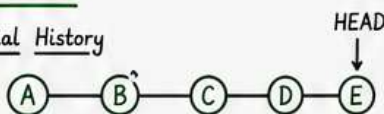
SCENARIO: How to recover a deleted commit using `git reflog`?

ANSWER

Even if a commit seems lost, `git reflog` keeps track of where HEAD and branch references were. You can use it to find and recover lost commits.

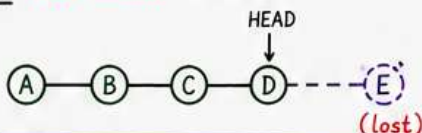
EXAMPLE

Initial History



You did `reset --hard` and lost last commit (E).

After `reset --hard` HEAD~1

TIP

- `reflog` is local to your machine.
- It has expiry (default ~ 90 days).
- Works for commits, checkouts, resets, merges etc.

SOLUTION

- 1 Check reflog
\$ `git reflog`
shows all actions
- 2 Find the commit hash of the lost commit (E) from reflog.
- 3 Go back to that commit (safe checkout)
\$ `git checkout <hash-of-E>`
or
\$ `git reset --hard <hash-of-E>`
move HEAD back to E
- 4 Now your commit is back!



Sample reflog output

HEAD@{n}	Action	Ref
HEAD@{0}	reset: moving	HEAD~1
HEAD@{1}	commit: add file	E
HEAD@{2}	commit: update	D
HEAD@{3}	checkout: moving	main
HEAD@{4}	commit: initial	C
...		

NOTE

`reflog` is powerful but use `--hard` with care. It can overwrite uncommitted changes.

COMMAND QUICK VIEW

`git stash`
→ save uncommitted changes

`git stash pop`
→ apply and remove stash

`git reflog`
→ show history of actions

`git reset --hard <hash>`
→ move HEAD to commit

11

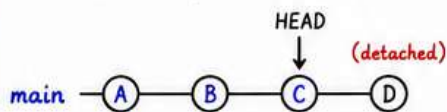
SCENARIO: What is detached HEAD and how do you get out of it?

ANSWER

Detached HEAD means your HEAD is pointing directly to a commit, **not** to any branch. You are **not** on a branch. Commits you make can be **lost** unless you create a branch.

EXAMPLE

You checkout a specific commit (C).



You are here (not on any branch).

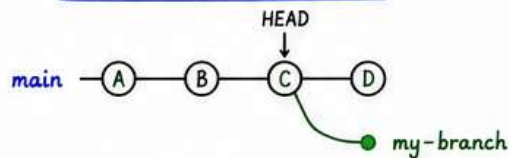
TIP

- Create a branch if you want to keep your work.
- Otherwise, your commits may be **garbage collected**.
- Use detached HEAD for testing or quick checkouts.

HOW TO GET OUT OF DETACHED HEAD?

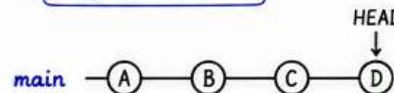
- ① Create a new branch from current commit

```
$ git switch -c my-branch # or
$ git checkout -b my-branch
```



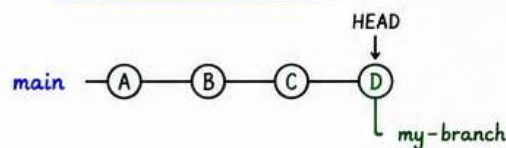
- ② Or switch back to any existing branch

```
$ git switch main
```



- ③ Or just create the branch and continue

```
$ git branch my-branch
$ git switch my-branch
```

 NOTE

If you make commits in detached HEAD and don't create a branch, they can be lost later!

QUICK RECAP

Checkout specific commit (directly)

↓
Detached HEAD
↓
Create branch or switch back
↓
Safe again! ✓

12

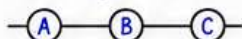
SCENARIO: What is the difference between `git push --force` and `git push --force-with-lease`?

ANSWER

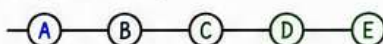
- Both **rewrite history** on remote.
- `--force` is dangerous.
- `--force-with-lease` is safer.

EXAMPLE

Remote (origin/main)



You (local main)



You want to push your changes.

① git push --forceWhat happens?

- Forces push and overwrites remote no matter what.
- It can **delete others' work** without warning.

Remote (overwritten 🤖)

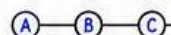
 NOTE

Use only when you are 100% sure no one else has pushed.

② git push --force-with-leaseWhat happens?

- Pushes only if remote is still at the commit you expect.
- If someone else pushed, it will **reject** the push.

Remote (unchanged)



After push (safe ✓)



If someone else pushed

Remote (updated by other)



Push rejected X
(Protects others' work)

KEY POINTS

- ✓ Prefer `--force-with-lease` always.
- ✓ It protects your team.
- ✓ `--force` is risky. Use with caution.

COMMAND QUICK VIEW

```
git push --force # overwrite remote (risky!)
git push --force-with-lease # safer way to push
```



13

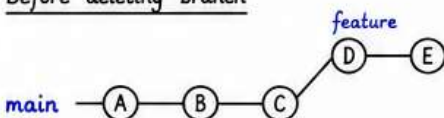
SCENARIO: How to recover a deleted branch ?

ANSWER

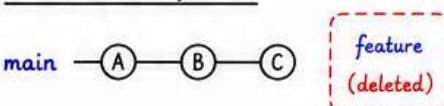
Even if you delete a branch, Git keeps a reference to it in **reflog** for some time. You can recover it.

EXAMPLE

Before deleting branch



Delete branch 'feature'

**TIP**

- Git **reflog** is your best friend!
- By default, reflog entries are kept for 90 days.
- After that, they may be garbage collected.

SOLUTION

- 1 Check reflog to find the last commit of the deleted branch.

```
$ git reflog
```

shows all actions

```

abcd123 HEAD@{0}: checkout: moving from feature to main
efgh456 HEAD@{1}: commit: add feature E
ijkl789 HEAD@{2}: checkout: moving from main to feature
mnop000 HEAD@{3}: branch: Created from main
  
```

- 2 Find the commit hash of the last commit on 'feature'.

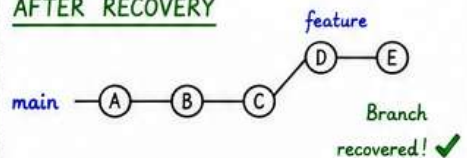
- 3 Recreate the branch pointing to that commit.

```
$ git branch feature efgh456
```

use the commit hash

- 4 Switch to the branch.

```
$ git checkout feature
```

AFTER RECOVERY**NOTE**

If reflog entry is too old and removed, recovery may not be possible.



14

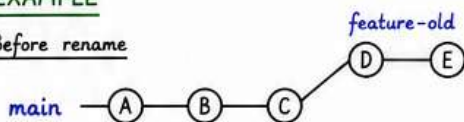
SCENARIO: How to rename a branch safely ?

ANSWER

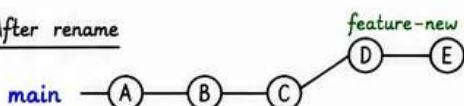
Use **git branch -m** to rename a branch. Update remote as well.

EXAMPLE

Before rename



After rename

**STEPS**

- 1 Rename the branch locally

```
$ git branch -m feature-old feature-new
```

or: **git branch -m feature-new**

- 2 Push the renamed branch to remote

```
$ git push origin feature-new
```

- 3 Delete the old branch from remote

```
$ git push origin --delete feature-old
```

- 4 (Optional) Set upstream

```
$ git push -u origin feature-new
```

VISUAL FLOW

Rename locally



Push new branch



Delete old remote



Set upstream (optional)



Done! ✓

TIP

- Use **-m** (move) option, not **-M**.
- Always update remote repository too.
- Inform your team about the change.

COMMON COMMANDS QUICK VIEW

- | | |
|--|--|
| • Rename branch (local) | • Push new branch |
| : git branch -m old-name new-name | : git push origin new-name |
| • Rename branch (local) | • Delete old branch (remote) |
| : git branch -m new-name | : git push origin --delete old-name |
| • Delete branch (local) | • Set upstream |
| : git branch -d old-name | : git push -u origin new-name |

15

SCENARIO: What is Git Worktree and how is it useful ?

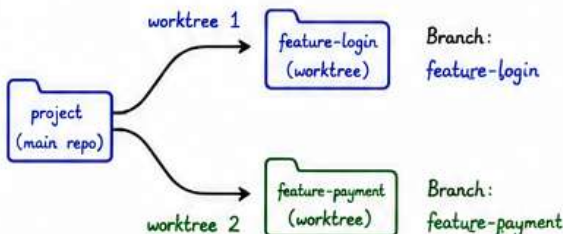
ANSWER

Git Worktree allows you to have multiple working directories attached to different branches of the same repository at the same time.

It is lighter and faster than cloning the repo multiple times.

EXAMPLE

You are working on two features simultaneously from the same repository.

TIP

- Worktree shares the same .git directory.
- Saves disk space (no need to clone again).
- Great for working on multiple branches in parallel.

WHAT IS GIT WORKTREE ?

- A worktree is an additional working directory linked to a specific branch.
- It has its own working files but shares the same Git object database.
- You can switch and work on multiple branches at the same time.

WHY USE WORKTREE ?

- ✓ Work on multiple branches simultaneously.
- ✓ No need to stash/commit and switch again and again.
- ✓ Faster than cloning multiple repositories.
- ✓ Keeps your main working directory clean.
- ✓ Perfect for feature development and hotfix in parallel.

HOW TO USE GIT WORKTREE ?

- 1 Check existing worktrees

```
$ git worktree list
```

shows all worktrees

- 2 Add a new worktree for a branch

```
$ git worktree add <path> <branch>
```

creates new working directory for branch

- 3 Create worktree and new branch at once

```
$ git worktree add -b <new-branch> <path>
```

- 4 Remove a worktree

```
$ git worktree remove <path>
```

removes worktree

- 5 Prune (cleanup) dangling worktrees

```
$ git worktree prune
```

removes invalid worktree entries

PRACTICAL EXAMPLE

Let's say your main repo is: **project**

- 1 Current repository

```
project/ (main repo)
Branch: main
```

- 2 Create worktrees

```
$ git worktree add ../login feature-login
```

```
$ git worktree add -b feature-payment ../payment
```

- 3 Result

```

project/ (main) -----> Branch: main
../login/ (worktree 1) -----> Branch: feature-login
../payment/ (worktree 2) -----> Branch: feature-payment
  
```

OUTPUT EXAMPLE (git worktree list)

```

$ git worktree list
* d6f8a2c [main] /home/user/project
9ab12ef [feature-login] /home/user/login
2c3d4ef [feature-payment] /home/user/payment

* = current worktree
  
```

FOLDER STRUCTURE (Simplified)

```

project/ (main repo)
├── .git/ (shared)
├── file1.txt
├── file2.txt
├── ../login/ (worktree 1)
│   └── (same files, different changes)
├── ../payment/ (worktree 2)
│   └── (same files, different changes)
  
```

NOTE

- Each worktree has its own HEAD and index but shares object database.
- Do not delete worktree folders manually. Use 'git worktree remove'.
- If a branch is in use by a worktree, you cannot delete it.

COMMAND QUICK VIEW

git worktree list → list all worktrees	git worktree add <path> <branch> → add worktree for branch	git worktree add -b <branch> <path> → add & create new branch	git worktree remove <path> → remove worktree	git worktree prune → cleanup invalid entries
---	---	--	---	---

16

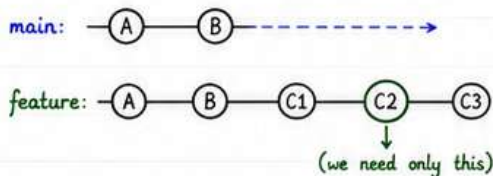
SCENARIO: What is git cherry-pick and when would you use it ?

ANSWER

Git cherry-pick lets you apply a specific commit from one branch onto another. It is useful when you need a particular change without merging the entire branch.

EXAMPLE

We have two branches: **main** and **feature**. Commit C2 on **feature** fixes a bug. We want only C2 on **main**, not other commits.

TIP

- Cherry-pick is safe and does not affect other commits.
- Always test after cherry-pick.
- Resolve conflicts if any.

HOW CHERRY-PICK WORKS ?

- Find the commit hash you want to cherry-pick.

```
$ git log --oneline
```

note the hash of C2

```
8f3a1c2 (HEAD -> feature) Fix: handle null case (C3)
b7d9e11 Improve validation logic (C2) ← we need this
3c4d5e6 Add new feature (C1)
1a2b3c4 (main) Update README (B)
0a1b2c3 Initial commit (A)
```

- Checkout the target branch.

```
$ git checkout main
```

- Cherry-pick the commit.

```
$ git cherry-pick b7d9e11
```

commit hash of C2

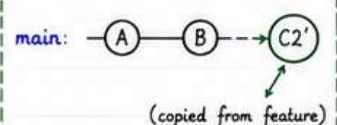
- If conflicts occur, resolve them, then:

```
$ git add <file>
```

```
$ git cherry-pick --continue
```

NOTE

- It creates a new commit (even if the change is same).
- Do not cherry-pick merge commits.
- Useful for hotfixes and quick fixes.

AFTER CHERRY-PICK

17

SCENARIO: What is git revert and when should you use it ?

ANSWER

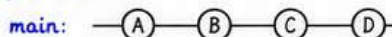
Git revert creates a new commit that undoes the changes of a previous commit.

It is used when you want to keep history intact.

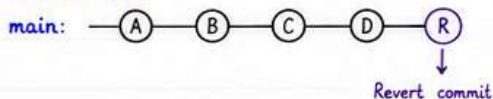
EXAMPLE

Commit B introduced a bug, so we revert it.

Before revert



After revert of B

TIP

- Revert is safe for shared branches (like main).
- It does not delete history.
- Preferred over reset on public branches.

HOW REVERT WORKS ?

- Find the commit you want to revert.

```
$ git log --oneline
```

note hash of commit B

- Revert the commit.

```
$ git revert <hash-of-B>
```

creates a new revert commit

- Resolve conflicts if any.

```
$ git add <file>
```

```
$ git revert --continue
```

VISUAL FLOW

Make changes
(commit B)

→

Revert it
(new commit R)

→

History intact
(B + R)

REVERT vs RESET (Quick Comparison)

Feature	git revert	git reset
Removes changes ?	No (adds new commit)	Yes (moves HEAD)
History	Preserved	Rewritten
Safe on shared branch	Yes	No (dangerous)
Use case	Undo commit	Undo local changes

QUICK RECAP

```
Find commit
↓
git revert <hash>
↓
Resolve (if any)
↓
History safe! ✓
```

COMMAND QUICK VIEW

```
git log --oneline
→ Show commit history
```

```
git cherry-pick <hash>
→ Apply specific commit
```

```
git checkout <branch>
→ Switch branch
```

```
git revert <hash>
→ Undo commit safely
```

18

SCENARIO: What is git stash and when would you use it ?

ANSWER

Git stash saves your uncommitted changes temporarily so you can work on something else, then come back and reapply them later.

EXAMPLE

You are working on a feature but another urgent bug comes up.

Before stash (changes not committed)

main: — A — B — C — [Uncommitted changes]

After stash

main: — A — B — C

TIP

- Stash is like a "shelf" for your changes.
- Your working directory becomes clean.
- You can apply the changes back anytime.

HOW GIT STASH WORKS ?

① See what changes you have

\$ git status # shows modified files

② Stash your changes

\$ git stash # saves changes and cleans working dir

Optional: Give a message

\$ git stash push -m "WIP: login feature"

③ Work on something else (switch branch, fix bug, etc.)

\$ git checkout bugfix-123

④ See your stashes

\$ git stash list # list all stashes

⑤ Apply stash back

\$ git stash apply stash@{0} # apply without removing

OR

\$ git stash pop stash@{0} # apply and remove

⑥ (Optional) Drop a stash

\$ git stash drop stash@{0} # remove stash

STASH LIST EXAMPLE

stash@{0}: WIP: login feature
stash@{1}: WIP: UI changes
stash@{2}: temp fixes

NOTE

Stash is local to your machine, not shared with others.

19

SCENARIO: How to undo a commit that is not pushed yet ?

ANSWER

If the commit is not pushed, you can undo it safely using reset or revert.

BEFORE (latest commit C not pushed)

main: — A — B — C — (latest)

TIP

- reset moves the branch pointer.
- revert creates a new commit that undoes the changes.
- Use reset only if not pushed.

OPTION 1: git reset (recommended)

Moves the branch pointer back and removes the last commit.

To undo last commit, keep changes:

\$ git reset --soft HEAD~1

To undo last commit, unstage changes:

\$ git reset --mixed HEAD~1

(default)

To undo last commit, discard changes:

\$ git reset --hard HEAD~1

Use --hard carefully. Changes will be lost!

OPTION 2: git revert

Creates a new commit that undoes the last commit. History remains intact.

\$ git revert HEAD

Result:

A — B — C — R —
(revert of C)

Use revert when:

- You have already pushed.
- You want to keep history clean.

QUICK COMPARISON

Method	Command	Moves branch pointer?	Creates new commit?	Safe if pushed?	When to use
reset --soft	git reset --soft HEAD~1	Yes	No	No	Keep changes staged
reset --mixed	git reset --mixed HEAD~1	Yes	No	No	Keep changes unstaged
reset --hard	git reset --hard HEAD~1	Yes	No	No	Discard all changes
revert	git revert HEAD	No	Yes	Yes	Undo safely (even if pushed)

COMMAND QUICK VIEW

git status → see changes	git stash → save changes	git stash list → list stashes	git stash apply → apply stash	git stash pop → apply & remove	git stash drop → remove stash
-----------------------------	-----------------------------	----------------------------------	----------------------------------	-----------------------------------	----------------------------------



20

SCENARIO: What is git merge and when would you use it ?

ANSWER

Git merge combines the changes from one branch into another.

It is used to bring together different lines of development.

EXAMPLE

We have a feature branch that is complete. We want to merge it into main.

Before merge

main: — A — B — C —

feature: — C — D — E —

After merge

main: — A — B — C — D — E —
(feature merged into main)

TIP

- Keep your main branch clean and stable.
- Always test before merging.
- Prefer pull request + code review before merge.

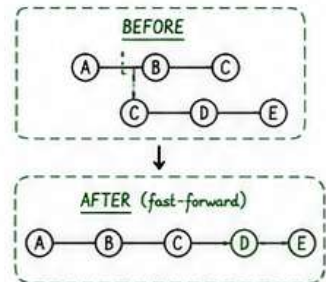
TYPES OF MERGE

① Fast-forward merge (linear history)

When target branch has not moved since the branch was created.

```
$ git checkout main
$ git merge feature
```

no new commit created

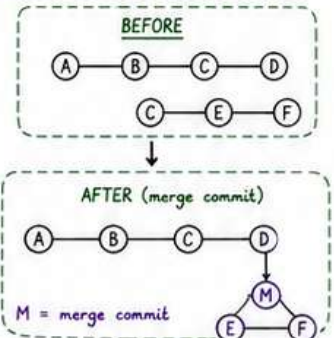


② Three-way merge (merge commit created)

When both branches have new commits.

```
$ git checkout main
$ git merge feature
```

creates merge commit

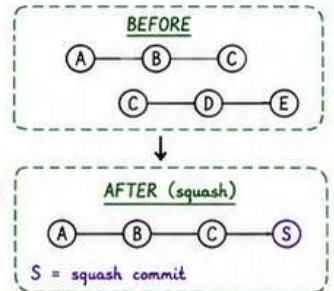


③ Squash merge (keep history clean)

Combine all commits from source branch into one commit when merging.

```
$ git checkout main
$ git merge --squash feature
$ git commit -m "Add feature"
```

one new commit

NOTE

- Merge preserves history (default behavior).
- Squash is useful for small/clean commits.
- Rebase is another option (not a merge).

MERGE vs REBASE (Quick)

Merge → keeps all history (safe)
 Rebase → rewrites history (clean)
 Use merge on shared branches.
 Use rebase on personal branches.

21

SCENARIO: What is conflict in Git and how do you resolve it ?

ANSWER

A conflict occurs when Git cannot automatically merge changes because the same part of a file was modified differently in two branches.

EXAMPLE

Both branches changed line 5 of file.txt

main changed:

Line 5: price = 100

VS

feature changed:

Line 5: price = 150

HOW TO RESOLVE CONFLICT ?

① Try to merge (conflict will occur)

```
$ git checkout main
$ git merge feature
```

② Git shows conflicting files

```
$ git status
```

③ Open the file and resolve conflicts
Edit the file and keep correct changes.

```
file.txt
Line 4: product = "Book"
<<<<< HEAD (main)
       price = 100
=====
       price = 150
>>>>> feature
Line 6: inStock = true
```

④ After editing, mark as resolved

```
$ git add file.txt
```

⑤ Complete the merge

```
$ git commit
```

TIP

- Resolve conflicts carefully.
- Test after resolving.
- Communicate with team to avoid conflicts.

NOTE

Conflicts are normal in team work. Stay calm and resolve step by step.



git merge <branch>
→ Merge branch

git merge --no-ff <branch>
→ Create merge commit

COMMAND QUICK VIEW
git merge --squash <branch>
→ Squash merge

git status
→ See conflicts

git add <file>
→ Mark resolved

git commit
→ Complete merge



22

SCENARIO: How to **rebase** a branch and when should you use it ?

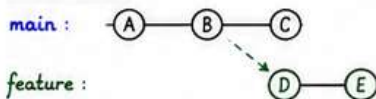
ANSWER

Git rebase moves or combines a sequence of commits to a new base commit. It creates a linear history.

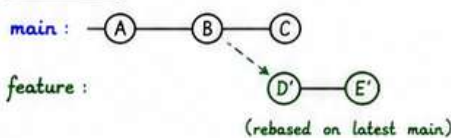
EXAMPLE

Feature branch is behind main. We want to bring it up to date.

Before rebase



After rebase

**TIP**

- Use rebase to keep history clean and linear.
- Do **NOT** rebase commits that are already pushed and shared.
- Rebase is great for personal / feature branches.

HOW TO REBASE ?

- 1 Switch to your feature branch
\$ git checkout feature
- 2 Fetch latest changes from main
\$ git fetch origin
- 3 Rebase your branch on latest main
\$ git rebase origin/main # replay your commits on top of latest main
- 4 Resolve conflicts if any
\$ git add <file>
\$ git rebase --continue # repeat until done
- 5 Push the rebased branch
\$ git push --force-with-lease # safe force push

REBASE vs MERGE (Quick Comparison)

	Rebase	Merge
History	Linear	Non-linear (merge commit)
Best for	Your feature branch	Shared / public branches
Commit IDs	Changes (new IDs)	Preserves all commit IDs
Safe without force push	No (may need force push)	Yes

WHEN TO USE REBASE ?

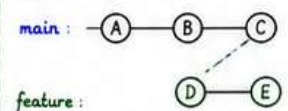
- ✓ When working on feature branches.
- ✓ To keep project history clean.
- ✓ Before creating a PR / MR.
- ✓ When commits are not shared yet.

**NOTE**

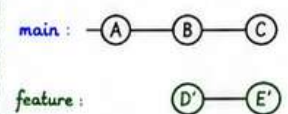
- Never rebase commits that are already pushed to shared branches.
- Rebase changes commit IDs.
- After rebase, you may need to force push.

**VISUAL FLOW**

Before rebase



After rebase



23

SCENARIO: How to **squash** multiple commits into one ?

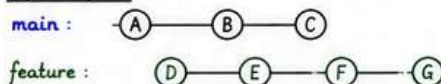
ANSWER

Git squash combines multiple commits into a single commit. Useful to clean up noisy or unnecessary commit history.

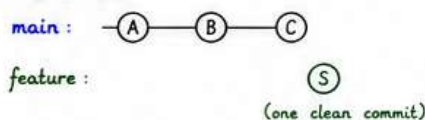
EXAMPLE

Feature branch has multiple WIP commits. We want to squash them into one.

Before squash



After squash

**HOW TO SQUASH ?**

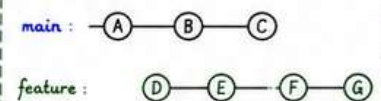
- 1 Interactive rebase to the commit before the first commit you want to squash.
\$ git rebase -i C # C is the commit before D
- 2 In the editor, mark commits
pick d1a2b3d D: add login
squash e4f5g6h E: fix login bug
squash f7g8h9i F: update UI
g g9h0i1j G: change text
 - Keep the first commit as pick
 - Change others to squash (or s)
 - Save and exit
- 3 Write a commit message for the squashed commit and save.
- 4 Push the squashed branch
\$ git push --force-with-lease

**TIP**

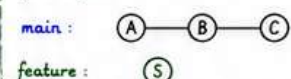
- Great for cleaning up WIP commits.
- Do it before pushing the branch.
- Keep meaningful commits.

VISUAL FLOW

Before squash



After squash

**NOTE**

- Squash rewrites history.
- Avoid on public/shared branches.
- Coordinate with team if needed.

**COMMAND QUICK VIEW**

git rebase -i <commit> → start interactive rebase	pick (keep commit) → commit stays	squash (or s) → combine with above commit	edit → modify commit	save & exit → apply changes	git push -f (safe) → update remote
--	--------------------------------------	--	-------------------------	--------------------------------	---------------------------------------



24

SCENARIO: How to handle uncommitted changes while switching branches?

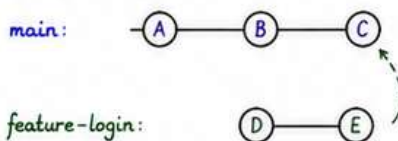
ANSWER

If you have uncommitted changes and want to switch branches, you have multiple safe options.

EXAMPLE

You are on feature-login but have some local changes not committed.

Current state

TIP

- Never lose your work.
- Choose the option based on your situation.
- Stash is the most flexible.

NOTE

- Avoid using `reset --hard` unless sure.
- Stash is safe and doesn't affect history.
- Temporary commit is useful for teamwork.

WAYS TO HANDLE UNCOMMITTED CHANGES

① Stash the changes (Recommended)

`$ git stash` # saves changes and cleans working dir

Switch branch → do work → apply stash

`$ git stash pop` # reapply changes

② Commit the changes (if meaningful)

`$ git add .`
`$ git commit -m "WIP: saving work"` # temporary commit

③ Stash with message (better tracking)

`$ git stash push -m "WIP: login UI"` # add message

④ Switch with stash in one step

`$ git stash push -m "WIP" && git checkout <branch>` # stash and switch

⑤ Discard changes (if not needed)

`$ git restore .` # discard all changes

or

`$ git reset --hard` # careful! removes everything

STASH OPTIONS

`$ git stash list` # list all stashes
`$ git stash show` # see latest stash
`$ git stash drop` # delete latest stash
`$ git stash clear` # delete all stashes

BEST PRACTICES

- ✓ Commit small and meaningful changes
- ✓ Write clear and descriptive commit messages
- ✓ Pull before you push
- ✓ Resolve conflicts carefully
- ✓ Keep your main branch clean
- ✓ Use branches for features / bugs
- ✓ Review before merging
- ✓ Keep history clean (rebase / squash)
- ✓ Communicate with your team
- ✓ Test your code before pushing

GIT COMMIT MESSAGE TIPS

Good: `fix: correct login validation`
`feat: add forgot password option`
`refactor: simplify payment logic`

Avoid: `update code`
`changes`
`fixed stuff`

Be clear and specific 😊

USEFUL COMMAND SUMMARY

TASK	COMMAND	WHAT IT DOES
Check status	<code>git status</code>	Shows modified, staged and untracked files
Add file	<code>git add <file></code>	Stages file for commit
Commit	<code>git commit -m "msg"</code>	Creates a new commit
Push	<code>git push origin <branch></code>	Pushes changes to remote
Pull	<code>git pull origin <branch></code>	Fetch + merge from remote
Branch	<code>git branch <name></code>	Creates new branch
Switch	<code>git checkout <branch></code>	Switch to branch
Merge	<code>git merge <branch></code>	Merge branch into current
Rebase	<code>git rebase <branch></code>	Reapply commits on top
Stash	<code>git stash</code>	Save uncommitted changes

COMMAND QUICK VIEW

`git status` → see status `git add .` → stage all `git commit -m "msg"` → commit `git pull origin` → pull latest `git push origin` → push `git branch` → list branches `git stash` → save changes `git log --oneline` → short history

25

SCENARIO: Any final tips for using Git like a pro?

ANSWER

Git is not just about commands, it's about understanding the flow and building good habits.

BEST PRACTICES

- ✓ Commit small and meaningful changes
- ✓ Write clear commit messages
- ✓ Pull latest changes before you start
- ✓ Use feature branches for new work
- ✓ Keep your history clean (rebase/squash)
- ✓ Test your code before pushing
- ✓ Review your code and others
- ✓ Communicate with your team
- ✓ Don't commit sensitive data
- ✓ Backup important branches

TIP

- Better git habit = Better collaboration 😊
- Slow and clean is better than fast and messy.
- Understand the **WHY**, not just the **HOW**.

COMMON MISTAKES

- ✗ Committing large or unrelated changes
- ✗ Pushing directly to main
- ✗ Not pulling before pushing
- ✗ Ignoring merge conflicts
- ✗ Force pushing on shared branches
- ✗ Bad commit messages
- ✗ Committing secrets or config files

COMPLETE GIT WORKFLOW (SUMMARY)

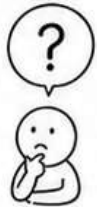
★ Repeat this cycle for every change you make. Keep it clean and consistent!

GIT GOLDEN RULES

- ① Never commit on **main**.
- ② One feature = One branch.
- ③ Commit often, but small.
- ④ Pull before push.
- ⑤ Resolve conflicts immediately.
- ⑥ Don't force push shared branches.
- ⑦ Keep history clean and readable. ★

WHEN IN DOUBT ?

- Check git status
- Read the error message carefully
- Google it (seriously 😊)
- Ask your team
- Practice more!

MOST USED GIT COMMANDS (CHEAT SHEET)

```
git status
git add <file>
git commit -m "msg"
git log --oneline
git branch
git checkout <branch>
git merge <branch>
git pull origin <branch>
git push origin <branch>
git stash
git stash pop
git rebase <branch>
git reset --hard <commit>
```

```
See the current status
Stage a file
Commit with message
See commit history (short)
List branches
Switch branch
Merge branch
Pull latest changes
Push changes
Stash your changes
Apply stashed changes
Rebase your branch
Reset to a commit (careful!)
```

REMEMBER

- Git is a superpower.
- The more you practice, the more confident you will become.
- Every pro was once a beginner. Keep going!

LEARN MORE

- Official Docs : git-scm.com/doc
- Pro Git Book : git-scm.com/book
- GitHub Skills : skills.github.com
- Git Cheat Sheet : education.github.com/git-cheat-sheet



Master Git and you will never be afraid of any version control challenge in interviews or real life.

Practice. Understand. Build. Repeat. You've got this!



SAVE

• SHARE

• FOLLOW @abhi_techhub

